ДИСЦИПЛИНА	Технологии индустриального программирования
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическое занятие
ПРЕПОДАВАТЕЛЬ	Адышкин Сергей Сергеевич
СЕМЕСТР	1 семестр, 2025-2026 гг.

Практическое занятие №4

Маршрутизация с `chi` (альтернатива — `gorilla/mux`). Создание небольшого CRUD-сервиса «Список задач».

Цели:

1. Освоить базовую маршрутизацию HTTP-запросов в Go на примере роутера `chi`.
2. Научиться строить REST-маршруты и обрабатывать методы `GET/POST/PUT/DELETE`.
3. Реализовать небольшой CRUD-сервис «ToDo» (без БД, хранение в памяти).
4. Добавить простое middleware (логирование, CORS).
5. Научиться тестировать API запросами через `curl`/Postman/HTTPie.

Теоретическая часть

1. Понятие маршрутизации в веб-приложениях

Когда сервер получает HTTP-запрос, он должен понять, **какой обработчик** («handler») выполнить. Например:

- `GET /users` → получить список пользователей,
- `POST /users` → создать нового пользователя,
- `DELETE /users/42` → удалить пользователя с ID 42.

Такое сопоставление URL-пути и метода запроса с функцией-обработчиком называется **маршрутизацией (routing)**.

В стандартной библиотеке Go для этого используется `http.ServeMux`. Однако для создания сложных REST API его возможностей бывает мало: неудобно работать с параметрами (`/users/{id}`), группировать маршруты или подключать middleware.

Поэтому Go-сообщество активно применяет сторонние роутеры, наиболее популярные из них:

- **gorilla/mux** — мощный и гибкий роутер (поддерживает переменные в пути, регулярные выражения, middleware).
- **chi** — лёгкий, минималистичный и быстрый роутер с поддержкой вложенных маршрутов и middleware.

Оба подходят для учебных и промышленных проектов.

2. CRUD и его связь с REST API

CRUD (Create, Read, Update, Delete) — базовые операции над данными. В REST API они напрямую соотносятся с HTTP-методами:

Операция	**HTTP метод**	**Пример маршрута**
Create	POST	`POST /tasks` — создать задачу
Read	GET	`GET /tasks` — список задач; `GET /tasks/{id}` — конкретная задача

Операция	HTTP метод	Пример маршрута
Update	PUT/PATCH	`PUT /tasks/{id}` или `PATCH /tasks/{id}` — изменить задачу
Delete	DELETE	`DELETE /tasks/{id}` — удалить задачу

Таким образом, чтобы реализовать список задач, нам нужно организовать набор маршрутов, соответствующих CRUD-операциям.

3. Особенности gorilla/mux и chi

gorilla/mux

- Поддержка переменных маршрута: `r.HandleFunc("/tasks/{id}", getTask).Methods("GET")`.
- Возможность указывать условия (например, регулярки для параметров).
- Простое подключение middleware (через `router.Use`).
- Недостаток: чуть более «тяжёлый» и медленный по сравнению с chi, но для учебных задач это не критично.

chi

- Минималистичный, но быстрый роутер.
- Красивый синтаксис вложенных маршрутов:

```
r.Route("/tasks", func(r chi.Router) {
    r.Get("/", listTasks)
    r.Post("/", createTask)
    r.Get("/{id}", getTask)
})
```

- Middleware также подключаются через `r.Use()`.
- Хорош для микросервисов и проектов, где важна производительность.

4. Middleware в маршрутизации

Middleware — это функции-прослойки, которые выполняются до или после обработчика. Они позволяют централизованно реализовать:

- логирование запросов,
- аутентификацию/авторизацию,
- CORS,
- ограничение скорости (rate limiting),
- обработку ошибок.

Подключение middleware в gorilla/mux и chi устроено похоже: можно «обернуть» маршруты и применить общие правила ко всем запросам.

5. Структура CRUD-сервиса на Go

Типичный минимальный проект для списка задач (`/tasks`) включает:

- **Модель (struct)** — описание задачи (id, title, done).
- **Хранилище (in-memory или база данных)** — пока можно использовать `map`/`slice` для упрощения.
- **Handlers (обработчики)** для CRUD-операций.
- **Роутер (mux или chi)** для маршрутизации.
- **main.go** — запуск сервера.

Пример (chi, упрощённо):

```
type Task struct {
    ID    int    `json:"id"`
    Title string `json:"title"`
    Done  bool   `json:"done"`
}

r := chi.NewRouter()
r.Get("/tasks", listTasks)
r.Post("/tasks", createTask)
r.Get("/tasks/{id}", getTask)
r.Put("/tasks/{id}", updateTask)
r.Delete("/tasks/{id}", deleteTask)
```

6. Значение для backend-разработчика

Маршрутизация и CRUD — это **базовые навыки**. Любой веб-сервис строится вокруг них:

- интернет-магазины управляют товарами и заказами,
- соцсети — пользователями и постами,
- корпоративные системы — документами и задачами.

Без понимания маршрутизации и CRUD невозможно создавать промышленный REST API. Этот шаг связывает предыдущие практики (HTTP-сервер, структура проекта) с дальнейшими темами курса — работой с БД, аутентификацией, документацией APIи тестированием.

Практическая часть

Структура проекта

```
pz4-todo/
  go.mod
  main.go
  internal/
    task/
      model.go
      repo.go
      handler.go
  pkg/
    middleware/
      logger.go
      cors.go
```

Пошаговая практика

Шаг 1. Инициализация модуля и установка зависимостей

```
mkdir pz4-todo
cd pz4-todo
go mod init example.com/pz4-todo
go get github.com/go-chi/chi/v5
go get github.com/go-chi/chi/v5/middleware
```

(Если хотите попробовать gorilla/mux — `go get github.com/gorilla/mux`, но в этом ПЗ используем chi.)

Шаг 2. Модель и «хранилище» в памяти

`internal/task/model.go`

```
package task

import "time"

type Task struct {
  ID        int64     `json:"id"`
  Title     string    `json:"title"`
  Done      bool      `json:"done"`
  CreatedAt time.Time `json:"created_at"`
  UpdatedAt time.Time `json:"updated_at"`
```

```
    }
    internal/task/repo.go
    package task

    import (
      "errors"
      "sync"
      "time"
    )

    var ErrNotFound = errors.New("task not found")

    type Repo struct {
      mu    sync.RWMutex
      seq   int64
      items map[int64]*Task
    }

    func NewRepo() *Repo {
      return &Repo{items: make(map[int64]*Task)}
    }

    func (r *Repo) List() []*Task {
      r.mu.RLock()
      defer r.mu.RUnlock()
      out := make([]*Task, 0, len(r.items))
      for _, t := range r.items {
        out = append(out, t)
      }
      return out
    }

    func (r *Repo) Get(id int64) (*Task, error) {
      r.mu.RLock()
      defer r.mu.RUnlock()
      t, ok := r.items[id]
      if !ok {
        return nil, ErrNotFound
      }
      return t, nil
    }

    func (r *Repo) Create(title string) *Task {
      r.mu.Lock()
      defer r.mu.Unlock()
      r.seq++
      now := time.Now()
      t  :=  &Task{ID:  r.seq,  Title:  title,  CreatedAt:  now,
UpdatedAt: now, Done: false}
      r.items[t.ID] = t
      return t
    }
```

```
func (r *Repo) Update(id int64, title string, done bool)
(*Task, error) {
  r.mu.Lock()
  defer r.mu.Unlock()
  t, ok := r.items[id]
  if !ok {
    return nil, ErrNotFound
  }
  t.Title = title
  t.Done = done
  t.UpdatedAt = time.Now()
  return t, nil
}

func (r *Repo) Delete(id int64) error {
  r.mu.Lock()
  defer r.mu.Unlock()
  if _, ok := r.items[id]; !ok {
    return ErrNotFound
  }
  delete(r.items, id)
  return nil
}
```

Шаг 3. Handlers: JSON API

internal/task/handler.go

```
package task

import (
  "encoding/json"
  "net/http"
  "strconv"

  "github.com/go-chi/chi/v5"
)

type Handler struct {
  repo *Repo
}

func NewHandler(repo *Repo) *Handler {
  return &Handler{repo: repo}
}

func (h *Handler) Routes() chi.Router {
  r := chi.NewRouter()
  r.Get("/", h.list)         // GET /tasks
  r.Post("/", h.create)       // POST /tasks
  r.Get("/{id}", h.get)       // GET /tasks/{id}
  r.Put("/{id}", h.update)    // PUT /tasks/{id}
  r.Delete("/{id}", h.delete) // DELETE /tasks/{id}
  return r
}
```

```
    func (h *Handler) list(w http.ResponseWriter, r
*http.Request) {
      writeJSON(w, http.StatusOK, h.repo.List())
    }

    func (h *Handler) get(w http.ResponseWriter, r *http.Request)
{
      id, bad := parseID(w, r)
      if bad {
        return
      }
      t, err := h.repo.Get(id)
      if err != nil {
        httpError(w, http.StatusNotFound, err.Error())
        return
      }
      writeJSON(w, http.StatusOK, t)
    }

    type createReq struct {
      Title string `json:"title"`
    }

    func (h *Handler) create(w http.ResponseWriter, r
*http.Request) {
      var req createReq
      if err := json.NewDecoder(r.Body).Decode(&req); err != nil
|| req.Title == "" {
        httpError(w, http.StatusBadRequest, "invalid json: require
non-empty title")
        return
      }
      t := h.repo.Create(req.Title)
      writeJSON(w, http.StatusCreated, t)
    }

    type updateReq struct {
      Title string `json:"title"`
      Done  bool   `json:"done"`
    }

    func (h *Handler) update(w http.ResponseWriter, r
*http.Request) {
      id, bad := parseID(w, r)
      if bad {
        return
      }
      var req updateReq
      if err := json.NewDecoder(r.Body).Decode(&req); err != nil
|| req.Title == "" {
        httpError(w, http.StatusBadRequest, "invalid json: require
non-empty title")
```

```
        return
      }
      t, err := h.repo.Update(id, req.Title, req.Done)
      if err != nil {
        httpError(w, http.StatusNotFound, err.Error())
        return
      }
      writeJSON(w, http.StatusOK, t)
    }

    func (h *Handler) delete(w http.ResponseWriter, r
*http.Request) {
      id, bad := parseID(w, r)
      if bad {
        return
      }
      if err := h.repo.Delete(id); err != nil {
        httpError(w, http.StatusNotFound, err.Error())
        return
      }
      w.WriteHeader(http.StatusNoContent)
    }

    // helpers

    func parseID(w http.ResponseWriter, r *http.Request) (int64,
bool) {
      raw := chi.URLParam(r, "id")
      id, err := strconv.ParseInt(raw, 10, 64)
      if err != nil || id <= 0 {
        httpError(w, http.StatusBadRequest, "invalid id")
        return 0, true
      }
      return id, false
    }

    func writeJSON(w http.ResponseWriter, code int, v any) {
      w.Header().Set("Content-Type",          "application/json;
charset=utf-8")
      w.WriteHeader(code)
      _ = json.NewEncoder(w).Encode(v)
    }

    func httpError(w http.ResponseWriter, code int, msg string)
{
      writeJSON(w, code, map[string]string{"error": msg})
    }
```

Шаг 4. Middleware: логирование и CORS

pkg/middleware/logger.go

```
    package middleware

    import (
```

```
      "log"
      "net/http"
      "time"
     )

     func Logger(next http.Handler) http.Handler {
      return   http.HandlerFunc(func(w   http.ResponseWriter,   r
*http.Request) {
        start := time.Now()
        next.ServeHTTP(w, r)
        log.Printf("%s    %s    %s",    r.Method,    r.URL.Path,
time.Since(start))
      })
     }
     pkg/middleware/cors.go
     package middleware

     import "net/http"

     func SimpleCORS(next http.Handler) http.Handler {
      return   http.HandlerFunc(func(w   http.ResponseWriter,   r
*http.Request) {
        w.Header().Set("Access-Control-Allow-Origin", "*")
        w.Header().Set("Access-Control-Allow-Methods",
"GET,POST,PUT,DELETE,OPTIONS")
        w.Header().Set("Access-Control-Allow-Headers",  "Content-
Type, Authorization")
        if r.Method == http.MethodOptions {
             w.WriteHeader(http.StatusNoContent)
             return
        }
        next.ServeHTTP(w, r)
      })
     }
```

Шаг 5. `main.go` — сборка приложения

```
     package main

     import (
      "log"
      "net/http"

      "github.com/go-chi/chi/v5"
      chimw "github.com/go-chi/chi/v5/middleware"

      "example.com/pz4-todo/internal/task"
      myMW "example.com/pz4-todo/pkg/middleware"
     )

     func main() {
      repo := task.NewRepo()
      h := task.NewHandler(repo)
```

```
      r := chi.NewRouter()
      r.Use(chimw.RequestID)
      r.Use(chimw.Recoverer)
      r.Use(myMW.Logger)
      r.Use(myMW.SimpleCORS)

      r.Get("/health",     func(w     http.ResponseWriter,     r
*http.Request) {
       w.Write([]byte("OK"))
      })

      r.Route("/api", func(api chi.Router) {
       api.Mount("/tasks", h.Routes())
      })

      addr := ":8080"
      log.Printf("listening on %s", addr)
      log.Fatal(http.ListenAndServe(addr, r))
     }
```

Запуск:

```
go run .
```

Откройте: `http://localhost:8080/health` — должно вернуть `OK`.

Шаг 6. Проверка CRUD через curl/Postman

Создать

```
curl -X POST http://localhost:8080/api/tasks ^
  -H "Content-Type: application/json" ^
  -d "{\"title\":\"Выучить chi\"}"
```

Список

```
curl http://localhost:8080/api/tasks
```

Получить по id

```
curl http://localhost:8080/api/tasks/1
```

Обновить

```
curl -X PUT http://localhost:8080/api/tasks/1 ^
  -H "Content-Type: application/json" ^
  -d "{\"title\":\"Выучить chi глубже\",\"done\":true}"
```

Удалить

```
curl -X DELETE http://localhost:8080/api/tasks/1
```

Примечание для PowerShell: экранирование кавычек как в примерах выше. Для macOS/Linux используйте одинарные кавычки: `-d '{"title":"..."}'`.

Дополнительные задания

Валидация длины `title` (минимум 3, максимум 100 символов).

1. Пагинация списка: `GET /tasks?page=1&limit=10`.
2. Фильтр по `done`: `GET /tasks?done=true`.
3. Простое «сохранение» на диск (при старте читать JSON-файл, при изменениях — сохранять).
4. Версионирование API: `/api/v1/...`.

Требования к отчёту

1. **На Git** (ФИО, группа, тема ПЗ, дата).
2. **Цели работы** (коротко).
3. **Ход работы**:

- структура проекта (дерево каталогов);
- фрагменты кода роутера, middleware, обработчиков;
- примеры запросов (`curl`/Postman) и ответы (скриншоты/вставки JSON);
- объяснение, как обрабатываются ошибки и коды ответа.

4. **Результаты тестирования** (таблица: маршрут → запрос → ожидаемый/фактический ответ).
5. **Выводы** (что получилось, что было сложным, что улучшить).
6. **Приложение** (полные листинги при необходимости).

Контрольные вопросы

1. Чем отличается роутер стандартной библиотеки (`http.ServeMux`) от `chi`/`gorilla/mux`? Какие преимущества `chi`?
2. Чем отличаются `GET`, `POST`, `PUT`, `PATCH`, `DELETE` и как их корректно применять в REST?
3. Что такое middleware и какие типичные задачи им решают? Примеры.
4. Как правильно возвращать ошибки и коды статусов в JSON-API? Приведите примеры.
5. Какие подходы к хранению данных можно использовать в учебном CRUD-проекте и что поменяется в коде при переходе с памяти на БД?

Критерии оценивания

- **Работоспособность API (4 б)**: все пять операций CRUD, корректные коды статусов.
- **Качество маршрутизации (2 б)**: логичная структура `/api/tasks`, параметры пути, обработка ошибок `404/400`.
- **Middleware (1,5 б)** логирование запросов и CORS.
- **Отчёт (1,5 б)**: полнота, скриншоты/ответы, выводы.
- **Аккуратность кода (1 б)**: читаемость, модульная структура, именование.

Подсказки и типичные ошибки

- Не забывайте заголовок `Content-Type: application/json` при `POST/PUT`.
- Всегда проверяйте ошибки `json.Decoder` и валидируйте вход.
- Для `DELETE` корректный ответ — `204 No Content` без тела.
- В Windows PowerShell экранирование отличается от bash — используйте кавычки как в примерах.

Альтернатива: `gorilla/mux` (по желанию)

Если хотите, замените только часть маршрутизации:

```
r := mux.NewRouter()
r.HandleFunc("/api/tasks", h.list).Methods("GET")
r.HandleFunc("/api/tasks",
h.create).Methods("POST")
r.HandleFunc("/api/tasks/{id}",
h.get).Methods("GET")
r.HandleFunc("/api/tasks/{id}",
h.update).Methods("PUT")
r.HandleFunc("/api/tasks/{id}",
h.delete).Methods("DELETE")
```

Остальной код остаётся тем же.